

UNIVERSITATEA TEHNICĂ A MOLDOVEI
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică

LUCRAREA DE LABORATOR NR. 7 & 8

Three Small Programs (Linux) & Aceleași apeluri (xv6)

Disciplina: Sisteme de Operare

A efectuat:
student gr. FAF-242
Daniel-Ioan Strunga

Au verificat:
asist. univ. Patricia Reițman
asist. univ. Marius Dumitrașcu

Chișinău — 2026

Cuprins

1 Laboratorul 7: Linux (Processes, Redirection, Pipes) 2
1.1 Part 1. fork + exec + wait 2
1.2 Part 2. A zombie, on purpose 2
1.3 Part 3. Redirection: `ls -l > out.txt` 3
1.4 Part 4. A pipe: `ls /etc | wc -l` 3

2 Laboratorul 8: Aceleași apeluri în OS-ul tău (xv6) 5
2.1 Partea 1. `run`, în xv6 5
2.2 Partea 2. `pingpong` 5

3 Concluzie: Comparație Linux vs. xv6 7

1 Laboratorul 7: Linux (Processes, Redirection, Pipes)

1.1 Part 1. fork + exec + wait

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        printf("child: my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *) NULL);
        perror("exec");
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Răspunsuri (Observe Part 1):

1. Linia `perror("exec")` se execută doar dacă apelul `execlp` eșuează (cum se întâmplă cu programul inexistent `nosuchprog`). Când rulăm `ls`, funcția `exec` înlocuiește complet memoria procesului cu noul executabil, deci instrucțiunile de după el nu mai sunt atinse.

2. Copilul afișează la „my parent" PID-ul procesului care a apelat `fork()` (părintele său).

3. Pentru `ls`, statusul returnat este **0** (succes). Pentru `nosuchprog`, este **127** (codul pe care l-am setat noi în `exit(127)`).

1.2 Part 2. A zombie, on purpose

Pentru a genera un proces zombie, am introdus `sleep(20)` în părinte, în fișierul `zombie.c`, înainte ca acesta să apeleze `waitpid`.

Răspunsuri (Observe Part 2):

1. `ls` primește litera `Z` (Zombie/defunct) în coloana `STAT`. Acest lucru arată că a terminat execuția, dar părintele nu i-a preluat încă statusul de ieșire.

2. După 20 de secunde, linia dispare deoarece părintele iese din `sleep` și execută `waitpid(pid, &status, 0);`, eliberând complet procesul din memorie.

1.3 Part 3. Redirection: `ls -l > out.txt`

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {
        close(1);
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *) NULL);
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

Răspunsuri (Observe Part 3):

1. `open` a returnat **1**. Sistemul de operare alocă întotdeauna cel mai mic descriptor liber, iar noi tocmai îl eliberasem pe 1 prin `close(1)`.

2. Rezultatul nu mai apare pe ecran deoarece `ls` scrie implicit la descriptorul 1 (`stdout`). Deoarece descriptorul 1 indică acum către `out.txt`, datele merg direct în fișier.

3. Pentru a redirecta fluxul de erori, ar fi trebuit să închidem descriptorul **2** (`stderr`) prin apelul `close(2)`.

1.4 Part 4. A pipe: `ls /etc | wc -l`

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int p[2];
    pipe(p);

    if (fork() == 0) {
        close(1); dup(p[1]);
        close(p[0]); close(p[1]);
        execlp("ls", "ls", "/etc", (char *) NULL);
        exit(127);
    }
    if (fork() == 0) {
        close(0); dup(p[0]);
        close(p[0]); close(p[1]);
        execlp("wc", "wc", "-l", (char *) NULL);
```

```
20          exit(127);
21      }
22      close(p[0]); close(p[1]);
23      wait(NULL); wait(NULL);
24      return 0;
25  }
```

Răspunsuri (Observe Part 4):

1. Numărul returnat de executabilul `./pipeline` este identic cu cel al comenzii de shell `ls /etc | wc -l` (247 de linii).

2. Dacă lăsăm `close(p[1])` comentat în părinte, procesul `wc` nu se termină niciodată.

3. `wc` se blochează așteptând simbolul EOF (End-Of-File), pe care sistemul de operare îl trimite abia când *toate* procesele au închis capătul de scriere al pipe-ului.

2 Laboratorul 8: Aceleași apeluri în OS-ul tău (xv6)

2.1 Partea 1. `run`, în xv6

```
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
  printf("parent: my pid is %d\n", getpid());
  int pid = fork();

  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0) {
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);
    printf("exec failed\n");
    exit(1);
  }

  int status;
  wait(&status);
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

Răspunsuri (Observă Partea 1):

1. În ambele sisteme am folosit triada `fork()`, `exec()` și `wait()`. Logica funcționării și fluxul datelor sunt absolut identice, diferind doar anumite detalii ale bibliotecii standard.

2. „hello from the child" este printat de utilitarul `echo`, care a suprascris programul `run` prin intermediul lui `exec`.

3. Linux ascunde statusul într-un format criptat binar în biblioteca `<sys/wait.h>`, deci e nevoie de macro-ul `WEXITSTATUS`. xv6, fiind minimalist, întoarce direct valoarea întreagă exact așa cum a fost transmisă în `exit()`.

2.2 Partea 2. `pingpong`

```
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
  int p2c[2], c2p[2];
  char b;
  pipe(p2c); pipe(c2p);

  if(fork() == 0) {
    close(p2c[1]); close(c2p[0]);
```

```
11      read(p2c[0], &b, 1);
12      printf("%d: received ping\n", getpid());
13      write(c2p[1], &b, 1);
14      exit(0);
15    }
16
17    close(p2c[0]); close(c2p[1]);
18    write(p2c[1], "x", 1);
19    read(c2p[0], &b, 1);
20    printf("%d: received pong\n", getpid());
21
22    wait(0);
23    exit(0);
24  }
```

Răspunsuri (Observă Partea 2):

1. Un pipe este strict unidirecțional (flux de octeți de la `[1]` la `[0]`). Pentru un trafic bidirecțional (părinte → copil și copil → părinte), avem nevoie de două magistrale distincte.

2. Părintele este oprit temporar de funcția blocantă `read()` care așteaptă date din pipe, și ulterior de `wait(0)` care îl obligă să asiste la terminarea procesului copil.

3 Concluzie: Comparație Linux vs. xv6

Prin realizarea acestor două lucrări de laborator am demonstrat modul în care conceptele fundamentale de gestionare a proceselor rămân universale, indiferent de complexitatea sistemului de operare.

Deși am folosit aceleași abstracții (procese, descriptori de fișiere, pipe-uri), mediile prezintă diferențe notabile:

- **Complexitatea bibliotecii C:** Pe Linux, ne bazăm pe glibc, care oferă apeluri superioare precum `execlp()` (care caută automat executabilul în `$PATH`). În xv6 (un kernel de uz educațional strict), folosim apelul nativ brut `exec()`, care necesită căile exacte.

- **Procesarea statusului:** Linux, fiind compatibil cu standardele POSIX complexe, codifică motivele opririi unui proces (cum ar fi semnalele asincrone sau segfault-urile) în variabila de status returnată de `waitpid`, necesitând macro-uri ca `WEXITSTATUS`. xv6 ignoră complet semnalele complexe, limitându-se la transmiterea unui simplu număr întreg între `exit()` și `wait()`.

- **Similitudinea conductelor (Pipes):** Abstracția descriptorilor a rămas identică în ambele arhitecturi. `pipe()`, `dup()` și comportamentul blocant al citirii (`read`) se comportă exact la fel în ambele medii, validând arhitectura clasică UNIX: totul este tratat ca un flux continuu de octeți (byte stream).

În esență, xv6 ne-a permis să vedem cum arată apelurile native de sistem „dezbrăcate" de comoditățile unui sistem modern, consolidând înțelegerea procesului fundamental care stă la baza nașterii și comunicării proceselor într-un sistem de operare derivat din UNIX.